

Session 7 · Deck 7.A3

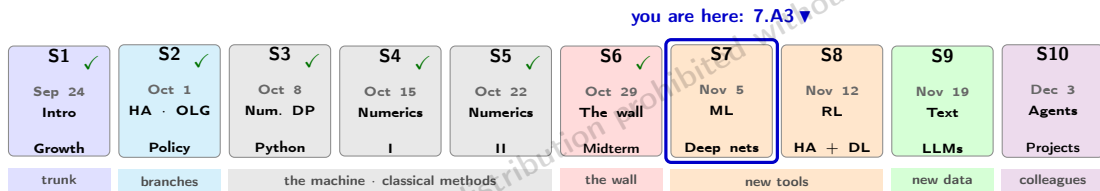
Training

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 7.A1 ML for macroeconomists · 7.A2 nets as approximators · **7.A3** training · 7.A4 architecture.

The Problem, Stated

7.A2 established that a network *can* represent the function.

This deck is about **finding** the parameters that do.

The objective is an empirical loss

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\mathcal{N}(\mathbf{x}_i; \theta), y_i),$$

and the contrast with everything in Sessions 3–5 is stark.

OLS has

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

— a closed form. $\mathcal{L}(\theta)$ has no closed form and is **not convex**.

Three questions, in order

1. How is $\nabla_{\theta} \mathcal{L}$ computed at all, for 10^5 parameters?
2. Which descent method, given that Newton is out?
3. When do you stop?

Non-convexity sounds fatal and is not. Multiple equilibria do not void comparative statics; multiple minima do not void prediction. What matters is *which* minimum, and we will get to that.

Forward Propagation

Computing the output is a recursion. With L hidden layers,

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{h}^{(\ell)} = \sigma(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}),$$

for $\ell = 1, \dots, L$, and then

$$\hat{\mathbf{y}} = W^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}.$$

Every layer does three things: multiply by a weight matrix, add a bias, apply the activation. Nothing else ever happens in the forward direction.

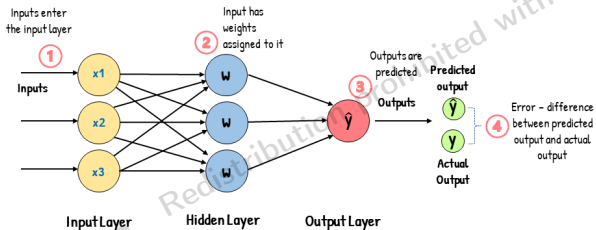
You have done this

A forward pass is **solving a recursive model forward in time**: given an initial state, apply the transition repeatedly and read off the terminal object. The only novelty is that the transition has 10^5 free parameters in it.

Note the last line has no σ . The output layer is affine for a regression; 7.A2's activation table says what it becomes for a classification.

The Forward Pass, Drawn

Feed-Forward Neural Network



Read it left to right: an input vector enters, each layer applies $\sigma(W \cdot +b)$, and a prediction leaves. **Every arrow is a multiplication by a number that will shortly be adjusted.**

A Classifier, on One Observation

The numbers on the next three frames come from the smallest honest version of 7.A1's digit classifier:

- **input** x — one feature of one image (in practice the flattened 784-vector);
- **hidden and output** — sigmoid units, so $\hat{y} \in [0, 1]$ is the predicted probability that the image is a “3”;
- **target** t — the true label, 1 if it is a 3 and 0 otherwise;
- **loss** L — squared error, for visibility; in practice cross-entropy.

Why one observation?

Because the arithmetic is **identical** to training on millions: one forward pass, one loss, one backward pass, one update — and then again. Everything that follows in this deck is about doing this faster, or stopping at the right time.

Forward, with Numbers

Input $x = 1.0$, one sigmoid hidden unit, one sigmoid output unit, target $t = 0$.

The network in full:

$$z_1 = w_1x + b_1, \quad h = \sigma(z_1)$$

$$z_2 = w_2h + b_2, \quad \hat{y} = \sigma(z_2)$$

$$L = \frac{1}{2}(\hat{y} - t)^2$$

Parameters $w_1 = 0.5$, $b_1 = 0$, $w_2 = 2.0$, $b_2 = 0$.

With the numbers:

$$z_1 = 0.5 \Rightarrow h = \sigma(0.5) \approx 0.62$$

$$z_2 = 1.24 \Rightarrow \hat{y} = \sigma(1.24) \approx 0.78$$

$$L = \frac{1}{2}(0.78)^2 \approx 0.30$$

Reading it:

- σ is a soft on/off switch: $z_1 = 0.5$ gives $h = 0.62$, so the hidden unit is “62% on”.
- $\hat{y} = 0.78$ is the predicted probability that this is a “3”.
- The truth is $t = 0$ — it is *not* a 3. The model is **confidently wrong**, and the loss is 0.30.
- Learning means nudging w_1, w_2 so that $\hat{y}$ moves toward t . **How much each moves** is what backpropagation computes.

Ledger `backprop_sigmoid.forward`. Code: [s07_generate.py](#), `run_backprop_sigmoid`; L07 Part 3.

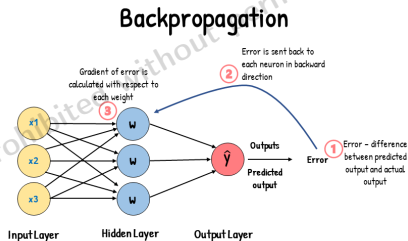
Backpropagation: Learning from a Mistake

Goal. $\partial L / \partial \theta$ for every parameter in the network.

Method. The chain rule, applied backwards through the layers — and *organised*, so that each intermediate quantity is computed once and reused.

The loop:

1. forward: all activations, and the loss;
2. backward: push the error signal from output to input;
3. read off $\partial L / \partial W$, $\partial L / \partial \mathbf{b}$;
4. update $\theta \leftarrow \theta - \eta \nabla_{\theta} L$.



4.A4 already made the case: reverse mode gives a whole gradient for about the cost of **one** function evaluation, however many inputs there are. That is why this is possible at all.

Backward, Part One: the Output Layer

Same example ($x = 1.0$, $t = 0$, $w_1 = 0.5$, $w_2 = 2.0$, $b_1 = b_2 = 0$); forward gave $h \approx 0.62$, $\hat{y} \approx 0.78$, $L \approx 0.30$.

Chain the links $w_2 \rightarrow z_2 \rightarrow \hat{y} \rightarrow L$:

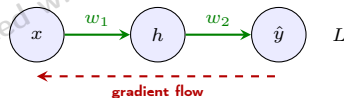
$$\frac{\partial L}{\partial w_2} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\hat{y}-t} \underbrace{\frac{\partial \hat{y}}{\partial z_2}}_{\hat{y}(1-\hat{y})} \underbrace{\frac{\partial z_2}{\partial w_2}}_h = 0.78 \times 0.17 \times 0.62 \approx 0.08$$

The first two factors are worth a name, because they will be reused:

$$\delta_2 \equiv \frac{\partial L}{\partial z_2} = (\hat{y} - t) \hat{y}(1 - \hat{y}) \approx 0.13,$$

so $\partial L / \partial w_2 = \delta_2 h$ and $\partial L / \partial b_2 = \delta_2 \approx 0.13$.

Ledger `backprop_sigmoid.backward`. **Code:** [s07_generate.py](#); [L07 Part 3](#).



fwd: compute $\hat{y}$, L

bwd: chain rule

Read the sign. $\partial L / \partial w_2 > 0$, so raising w_2 would *raise* the loss — descent lowers it. The output overshoot $t = 0$.

Backward, Part Two: One Layer Further, and the Update

Carry $\delta_2 \approx 0.13$ back through w_2 , then through the hidden activation:

$$\frac{\partial L}{\partial h} = \delta_2 w_2 \approx 0.26, \quad \frac{\partial h}{\partial z_1} = h(1 - h) \approx 0.24,$$

$$\delta_1 \equiv \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \cdot \frac{\partial h}{\partial z_1} \approx 0.06,$$

and finally $\partial L / \partial w_1 = \delta_1 x \approx 0.06$.

The update. Step against the gradient at **learning rate**

$\eta = 0.1$ — the one free choice in the algorithm,

$w \leftarrow w - \eta \partial L / \partial w$:

$$w_2 \leftarrow 2.0 - 0.1(0.0839) = 1.992,$$

and likewise $w_1 \leftarrow 0.494$.

Ledger backprop_sigmoid, checked against autograd. Code: [s07_generate.py](#); L07 Part 3.

That is the whole algorithm

Exactly what `loss.backward()` and one optimizer step do in 3.B3's five-line loop — at millions of parameters, and not otherwise different. And look at the two factors $\hat{y}(1 - \hat{y})$ and $h(1 - h)$: neither exceeds **0.25**. Every extra sigmoid layer multiplies the signal by one more of them — 7.A2's vanishing gradient, seen from the other end.

The Same Thing with ReLU, in Four Parameters

$x = 2$, $y = 3$, one ReLU hidden unit, $L = (\hat{y} - y)^2$, and $(w_1, b_1, v, c) = (0.5, -0.1, 1.2, 0.3)$.

Forward. $z = 0.9$, $h = 0.9$, $\hat{y} = 1.38$, $L = 2.6244$.

Backward.

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = -3.24, \quad \frac{\partial L}{\partial v} = -2.916,$$

$$\frac{\partial L}{\partial h} = -3.888, \quad \frac{\partial L}{\partial z} = -3.888, \quad \frac{\partial L}{\partial w_1} = -7.776.$$

Update at $\eta = 0.01$: $w_1^{\text{new}} = 0.5778$.

What changed, and what did not

The recipe is identical. Only $\partial h / \partial z$ differs: for ReLU it is 1 on the active side and 0 on the other. **No shrinkage factor.**

That single fact is most of why ReLU replaced the sigmoid in hidden layers.

And the zero is real: a unit whose input is negative for every sample receives no gradient at all and never recovers. That is the “dying ReLU”, and it is why 7.A2 listed Leaky ReLU.

Feng & Miao, *AI for Economists*, ch. 9 §9.3.2. Ledger backprop_worked; every figure reproduced by running it, agreeing with autograd to machine precision.

Code: [s07_generate.py](#), [run_backprop_worked](#).

The Recursion, in General

The forward pass ends in an affine output and a loss, and the error signal $\delta^{(\ell)} = \partial \mathcal{L} / \partial \mathbf{z}^{(\ell)}$ starts there:

$$\hat{\mathbf{y}} = W_{L+1} \mathbf{h}^{(L)} + \mathbf{b}_{L+1}, \quad \mathcal{L} = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2, \quad \delta^{(L+1)} = \hat{\mathbf{y}} - \mathbf{y}.$$

The rest is two lines:

$$\delta^{(\ell)} = (W_{\ell+1}^\top \delta^{(\ell+1)}) \odot \sigma'(\mathbf{z}^{(\ell)}),$$
$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \delta^{(\ell)} (\mathbf{h}^{(\ell-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_\ell} = \delta^{(\ell)}.$$

Read the first: transpose the *same* weight matrix used going forward, apply it to the signal above, multiply pointwise by the local slope. Read the second: the gradient at a weight is the signal on its output side times the activation on its input side.

Feng & Miao, *AI for Economists*, ch. 9, eqs. (9.20)–(9.21).

Two facts follow immediately. (1) σ' multiplies at every layer, so the activation choice controls whether the signal survives depth. (2) $\mathbf{h}^{(\ell-1)}$ is needed on the way back, so the **forward activations must be stored** — the memory cost of training, and why batch size is limited by the card.

Backpropagation Is the Adjoint Method

$\delta^{(\ell)}$ is the derivative of the objective with respect to an *intermediate quantity* — how much the loss would change if $\mathbf{z}^{(\ell)}$ were perturbed and everything downstream re-optimised.

That is a shadow price. It is the multiplier on the constraint “ $\mathbf{z}^{(\ell)}$ is what the previous layer produced”, and the backward recursion is exactly the costate equation run from the terminal condition backwards.

The technique has a name in optimal control: the **adjoint method**. Backpropagation is the adjoint method on a discrete computation graph.

Why this is worth a slide

You already know how to read a multiplier. δ large means that layer's output matters a great deal at the current parameters; $\delta \approx 0$ means it does not, and no amount of adjusting the weights below it will help. A *diagnosis*, not a metaphor: printing per-layer gradient norms is standard practice for exactly this reason.

1.B1's Lagrangian and 5.A1's implicit-function argument are the same mathematics, met a third time.

One Graph, Two Traversals — and What It Costs

The forward pass and the backward pass are two traversals of **one** object: the computation graph, whose nodes are tensors and whose edges are operations. 3.B3 built it; this is what it is for.

The cost follows from the structure. With P parameters, one sample:

- forward $\approx 2P$ floating-point operations (a multiply and an add per weight);
- backward $\approx 4P$ — the signal to the layer below, *and* the gradient at the weight;
- total $\approx 6P$ per sample per pass.

$$\text{FLOPs} \approx 6 P n E$$

for n samples and E epochs.

Use it as a budget

$P = 10^5$, $n = 5 \times 10^4$, $E = 100$ needs about 3×10^{12} FLOPs — seconds of real arithmetic. **A tabular macro network is not a big computation:** dozens of configurations are affordable without a GPU.

A chip's advertised **peak** FLOPS is never achieved — measured utilisation is 21.3% for GPT-3, 46.2% for PaLM-540B. A runtime worked out from peak is optimistic: **multiply it by two to four.**

Gradient Descent: the Engine

One update rule covers everything in this deck:

$$\theta_{n+1} \leftarrow \theta_n - \lambda_n g(b_n; \theta_n),$$

where g is the gradient computed on a batch b_n of size n :

- $n = 1$: **stochastic** gradient descent — one draw per step;
- $n = N$: **full** gradient descent — the whole sample per step;
- $1 < n < N$: **mini-batch** — what everyone actually does.

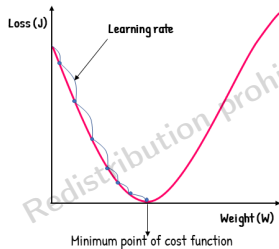
$$\frac{d\mathcal{L}(\theta)}{d\theta} \approx \frac{1}{n} \sum_{i=1}^n \frac{d\mathcal{L}(\theta; b_i)}{d\theta}$$

The algorithm, entire

1. initialise θ at random;
2. draw a mini-batch, form $g(b_n; \theta_n)$;
3. pick $\lambda_n > 0$ and step;
4. repeat.

This is 4.A3's M -sum. There the sampled gradient was introduced on a two-term objective; here the terms are observations, or sampled states. The object is the same.

Learning Rate



The picture is 4.A3's, redrawn: step against the gradient, with the step length the only real decision. What the picture cannot show is that this surface has 10^5 axes, which is the subject of the next two frames.

Why Not Newton — 4.A3 Already Answered This

Session 4 spent a block on second-order methods and closed with the frame that settles this one. A Hessian for N parameters has N^2 entries, and **numbers that must be stored** grow accordingly:

N	Newton	L-BFGS	CG
10^2	10^4	2,000	300
10^4	10^8	2×10^5	3×10^4
10^6	10^{12}	2×10^7	3×10^6

$N = 10^6$ is the parameter count of a *small* network.

The frame this cites

4.A3 put it plainly: 10^{12} Hessian entries is why **nobody trains a network with Newton's method**, and why Session 7's optimizer is a first-order one. That promise is now kept. Everything from here on uses gradients only.

Note that even L-BFGS with ten stored pairs needs 2×10^7 numbers at $N = 10^6$ — affordable, and still not what is used, for the reason on the next frame.

Ledger optimizer_memory from s04_numbers.json — cited, not recomputed. Code: [s04_generate.py](#).

And Why Not Conjugate Gradient Either

4.A3's other result was that conjugate gradient is **exact in at most N steps** on an N -dimensional quadratic, storing only a handful of vectors. On the two-dimensional example it reached the optimum from $(2, 2)$ in **two** steps, with the intermediate point $(0.889, -0.222)$.

So why is it not the method here? Because both CG and L-BFGS need a gradient that is **the same function every time they are asked**. Line searches and conjugacy corrections assume a fixed objective.

A mini-batch gradient is a **different** function at every step. The conjugacy that CG constructs is destroyed by the noise before it can be used.

The trade, stated once

Second-order and conjugate methods buy **accuracy per step**, and pay in memory and in the requirement of an exact gradient. Training a network wants the opposite: **very many cheap, noisy steps**. The noise is not merely tolerated — the next frames argue it helps.

The exception proves it: *full-batch* L-BFGS is a perfectly sensible choice for a small collocation problem, and is what a projection solver uses.

Ledger `cg_steps` from `s04_numbers.json` — cited, not recomputed. Code: [s04_generate.py](#).

Why a Sampled Gradient Still Gets There

Every loss here is a sum over terms, $\mathcal{L} = \frac{1}{N} \sum_i \ell_i$ — one per observation, in Block B one per sampled state — and **SGD differentiates one term at a time**. 4.A3 ran that on the smallest case: two terms, $\ell_1 = \frac{1}{2}x^2$, $\ell_2 = y^2$, from (2, 2) at step size $\lambda = 0.5$, full gradient (2, 4).

“Drawn” is the term the step used:

θ	drawn	next
(2, 2)	ℓ_2	(2, 0)
(2, 0)	ℓ_1	(1, 0)
(1, 0)	ℓ_1	(0.5, 0)
(0.5, 0)	ℓ_2	(0.5, 0)
(0.5, 0)	ℓ_1	(0.25, 0)

A coordinate whose term was not drawn does not move;

row four draws ℓ_2 at $y = 0$ and does nothing. The path is not monotone, and is not meant to be.

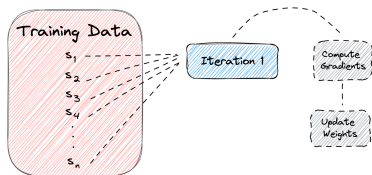
Ledger sgd_trajectory, sgd_dimension from s04_numbers.json — cited, not recomputed. Code: [s04_generate.py](#).

And why it still works

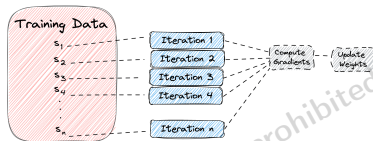
4.A3 measured how the noise scales with the **number of parameters** P — not the sample size. Its norm grows like $\sqrt{P}$: 1, 10, 100, 1,000 at $P = 1, 10^2, 10^4, 10^6$, while the *useful* component grows with the full gradient. The ratio degrades slowly enough that very many cheap steps beat few exact ones.

Mini-batches are that sum, and the schedule below is Robbins–Monro. Nothing in this optimizer is new; only its scale is.

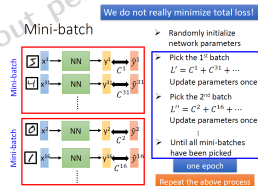
Batch, Stochastic, Mini-Batch



Batch. A smooth path, and every step expensive — it uses all the data.



Stochastic. A noisy, high-variance path; each step is cheap, and the noise can leave a poor minimum.



Mini-batch. Parallelisable like the first, averaged like the second. The practical standard.

The three pictures are the same descent with three answers to one question: **how many terms of the sum do you look at before you move?**

Why Mini-Batch? Four Reasons

1. **Computational efficiency.** Far more updates per pass over the data than full gradient descent, and so faster convergence in wall-clock time.
2. **Memory.** The activations of a full dataset do not fit on a card — and the previous frames showed they must be stored.
3. **Implicit regularisation.** The noise in the gradient estimate helps the iterate leave saddle points and shallow minima. It is doing statistical work, not just saving time.
4. **Parallelisation.** A batch is a matrix multiplication, which is what the hardware is for.

The econometric analogue

Mini-batch descent is sampling-based estimation: trade exactness for speed — and find that the noise is **helpful**, not merely tolerable.

In 7.B1 there is no dataset, so “batch” means **a fresh draw of states** each step. Nothing is ever reused, and reason (3) is doing most of the work.

Convergence: What Is Actually Guaranteed

Under a Lipschitz gradient and bounded gradient-noise variance, if the step sizes satisfy

$$\sum_{n=1}^{\infty} \lambda_n = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \lambda_n^2 < \infty,$$

then the weighted average of expected squared gradients vanishes:

$$\mathbb{E} \left[\frac{1}{\Delta_N} \sum_{n=1}^N \lambda_n \|\nabla F(\theta_n)\|^2 \right] \rightarrow 0, \quad \Delta_N = \sum_{n=1}^N \lambda_n.$$

In words: with the step size shrinking at the right rate, stochastic gradient descent settles at a **stationary point**.

Bottou, Curtis & Nocedal (2018), *Optimization Methods for Large-Scale Machine Learning*.

Read the statement carefully

It promises a *stationary point* — not a global minimum, not even a local one, and it says nothing about how long. Compare

3.A2's value-function iteration, where the contraction gave a **unique** fixed point and a computable error bound. **That is what is being given up**, and it is the honest price of the learned basis.

The Robbins–Monro Conditions, Read as Economics

These are the **Robbins–Monro** conditions (1951), the foundation of *stochastic approximation* — driving a noisy signal to its root. That is exactly what descent on a mini-batch gradient is.

$\sum \lambda_n = \infty$ — “**go the distance.**” The steps must sum to infinity so the iterate can travel *any* distance, however far the start. Decay too fast and the total path length is finite: training **stalls short**.

$\sum \lambda_n^2 < \infty$ — “**kill the noise.**” The squared steps must be summable so accumulated gradient noise stays finite and dies. This forces $\lambda_n \rightarrow 0$ fast enough that late steps stop bouncing and the iterate **settles**.

- **Concretely:** $\lambda_n = 1/n$ satisfies both. $\lambda_n = 1/\sqrt{n}$ satisfies the first and fails the second — too noisy. A **constant** λ also fails the second, and SGD then bounces forever in a ball around the optimum, which is exploration, not convergence.
- **4.A3 named this already.** The learning-rate schedule is not a hyperparameter with a folklore value; it is the condition under which the sampled-gradient recursion has a limit at all.

Adaptive Step Sizes: RProp $\rightarrow$ RMSProp $\rightarrow$ Adam

RProp (Riedmiller & Braun, 1993) sizes each parameter's step from the **sign** of its gradient alone. Robust — but the signs are too noisy once batches are sampled.

RMSProp (Hinton, 2012) adapts that to mini-batches: give each parameter its own step by dividing by a running average of *squared* gradients,

$$r_{k+1} \leftarrow \rho r_k + (1 - \rho) g_k \odot g_k,$$

$$\Delta\theta_{k+1} = -\frac{\lambda}{\sqrt{\delta + r_{k+1}}} \odot g_k,$$

damping steep directions and amplifying flat ones.

Adam (Kingma & Ba, 2015) is momentum *and* RMSProp: running averages of the gradient and of its square, with decay rates ρ_1, ρ_2 (defaults 0.9, 0.999).

What Adam is, in one line

Adam is stochastic gradient descent with a per-parameter step size. It is still first-order. It never forms, stores or inverts a Hessian — which is precisely the constraint the memory-wall frame imposed.

The economist's reading of momentum: an exponentially weighted average of past gradients is **adaptive expectations** over the descent direction.

Kingma & Ba (2015); Hinton (2012); Riedmiller & Braun (1993). Adam was the default choice from 2015 until roughly 2022.

Beyond Adam

- **AdamW** (Loshchilov & Hutter, 2019) — decoupled weight decay; the de-facto default.
- **Lion** (Chen et al., 2023) — a sign-based update, lower memory than Adam, competitive at large scale.
- **Sophia** (Liu et al., 2023) — a cheap second-order method using a *diagonal* Hessian estimate. Note what makes it affordable: the diagonal is N numbers, not N^2 .
- **Schedule-free** methods (Defazio et al., 2024) — drop the learning-rate schedule; the optimizer self-regulates.

Course default

`torch.optim.AdamW(lr=1e-3, weight_decay=1e-2)` for every lab in this course, varied only where a slide says so.

The economist's take: choosing an optimizer is choosing a numerical solver — it depends on the geometry of the problem. At the 10^3 – 10^6 parameter scale of macro networks, AdamW is the safe answer.

Loss I: Squared Error Is OLS

For a continuous target,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2,$$

the average squared gap, with big misses penalised quadratically.

Why this form and not another? Minimising it is **exactly maximum likelihood under Gaussian errors**; for a linear model it *is* OLS. The loss is not a convention — it encodes an assumption about the noise.

Change the noise model and the loss changes with it: Poisson for counts, pinball for quantiles.

Where this lands

GDP growth, yields, prices — and, in 7.B1, **Euler-equation residuals**. There the “target” is zero and the “prediction” is the residual, so minimising MSE is asking the equilibrium condition to hold. **No data enters.**

And 5.B5 already told you what that is: a *least-squares weighted residual*. The loss function of a neural solver has a name in Session 5.

Loss II: Cross-Entropy Is McFadden

A classifier chooses among K classes; the network emits a raw score z_k per class and **softmax** turns the scores into probabilities:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad \sum_k \hat{y}_k = 1.$$

The label is **one-hot**: y has a single 1 at the true class. A true “3” is $y = (0, 0, 0, 1, 0, \dots, 0)$.

Cross-entropy:

$$\mathcal{L}_{\text{CE}} = - \sum_k y_k \log \hat{y}_k = - \log \hat{y}_{k^*}.$$

Every term vanishes but the true one, so CE is **the negative log-probability assigned to the correct answer**. Minimising it maximises that probability: $\hat{y}_{k^*} = 0.99$ gives 0.01; $\hat{y}_{k^*} = 0.1$ gives 2.3. Confident and right costs nothing; confident and wrong is unbounded.

This is McFadden (1973). Softmax is the logit choice probability, the one-hot y is the observed choice, and minimising cross-entropy is maximising the multinomial-logit log-likelihood. **Classification is discrete choice.** You have estimated this model; you have not called it a loss function before.

McFadden (1973), “Conditional Logit Analysis of Qualitative Choice Behavior”. The binary toy above is the $K = 2$ case.

Terminology, and the Arithmetic Behind It

- **Epoch** — one complete pass through the training data.
- **Mini-batch** — the subset used for one update.
- **Iteration** — one parameter update, that is, one mini-batch.
- **Automatic differentiation** — the exact-gradient machinery behind `loss.backward()` (3.B3, 4.A4).
- **Episode** — a reinforcement-learning term: one path from an initial to a terminal state. It is a simulated life cycle, and it belongs to S8.

Step counts from `ledger.epoch_tradeoff`.

Do the arithmetic once

With $n = 50,000$ and batch 256, one epoch is 156 iterations. At 100 epochs that is about 15,600 updates. **“Epochs” is not the unit that matters — updates is.** The same 100 epochs on $n = 2,000$ is only about 600 updates, which is why a small sample needs far more epochs to reach the same place.

Overfitting, and When to Stop

The tension. Training loss falls essentially forever. Validation loss falls, then **turns up**. The turning point is overfitting becoming visible.

Stopping rules, in increasing order of sense:

1. a fixed number of epochs — simple, and arbitrary;
2. watch the validation loss and stop improving;
3. **early stopping** — halt after the validation loss has failed to improve for k epochs, and keep the best checkpoint.

Split the data **before** training, and standardise using training statistics only — otherwise the validation set has already leaked into the model.

Feng & Miao, *AI for Economists*, ch. 9 §9.3.4 on splitting and leakage.

The econometric name

Overfitting is data mining: the model memorises the noise in the sample rather than the process that generated it. Out-of-sample validation is the discipline, and it is the same discipline.

Leakage cannot be repaired by regularisation. If the split is wrong — a panel split across individuals, a time series split at random — no penalty will save the estimate.

Measured: What More Data Actually Buys

n	best ep.	best val.	final train	OLS
2,000	82	0.1535	0.0085	0.2879
10,000	53	0.1206	0.0362	0.3466
50,000	39	0.0788	0.0505	0.3273
<i>irreducible noise floor</i>			0.0625	

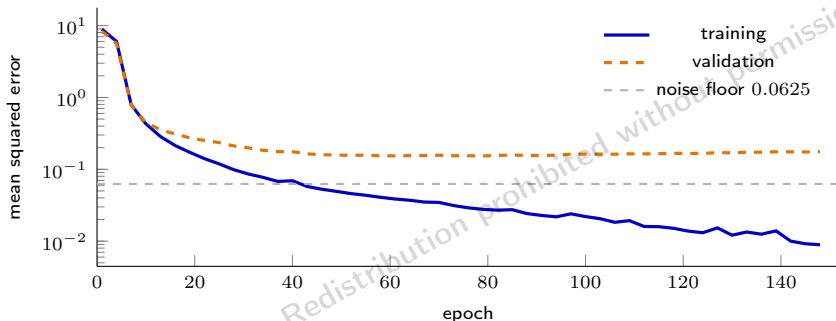
$d = 30$, of which 17 are pure noise; one interaction term and one tanh term. The last row is the **irreducible noise variance** — no method can beat it, and nothing honest can go below it.

Ledger epoch_tradeoff: median over three seeds, 80/20 split standardised on training statistics. Design follows Feng & Miao, *AI for Economists*, ch. 9 §9.3.4; the process here is our own and the numbers are this run's. Code: [s07_generate.py](#), `run_epoch_tradeoff`.

Read the *final train* column against the floor. At $n = 2,000$ the training loss is **0.0085** — **seven times below** 0.0625, a level no method can legitimately reach. The network is fitting noise it cannot possibly be learning, and that single comparison is the cleanest diagnostic of overfitting there is.

The gap to the floor closes with n : validation 0.1535 $\rightarrow$ 0.1206 $\rightarrow$ 0.0788, against a floor of 0.0625. And the best epoch falls, 82 $\rightarrow$ 53 $\rightarrow$ 39 — more data, fewer passes, because each pass is more updates.

The Same Run, as a Picture



$n = 2000$, one seed. The training curve goes **straight through the noise floor** and keeps falling; the validation curve bottoms out near epoch 82 and then drifts up. **Everything to the right of that minimum is memorisation**, and early stopping is the instrument that refuses to buy it.

Ledger fig_epoch_train, fig_epoch_val; plotted every third epoch. Code: [s07_generate.py](#), run_epoch_tradeoff.

Local Optima, and Why the Worry Faded

The loss surface is not convex: many local optima, many more saddle points, and the global optimum is generally unreachable.

What helps: random initialisation, momentum and adaptive steps to leave shallow basins, and the mini-batch noise as exploration.

The honest answer: the global optimum is neither necessary nor desirable. A good local optimum that *generalises* is enough.

Is this still the frontier?

No. The *benign landscape* result — in over-parameterised networks most local minima perform similarly (Choromanska et al., 2015) — is now standard. The live questions are **mode connectivity** and **why** over-parameterisation helps — which minima generalise, not whether good ones exist. The economist's version: multiple equilibria do not stop you doing comparative statics. They oblige you to say which one you are at.

Choromanska et al. (2015); Garipov et al. (2018); Frankle et al. (2020).

Three Instruments Against Overfitting

1. **Weight decay.** Add $\lambda \|\theta\|_p$ to the loss — $p = 1$ for sparsity (LASSO), $p = 2$ for shrinkage (Ridge). Biases are excluded. Typical $\lambda \in [10^{-5}, 10^{-2}]$. In the ℓ_2 case the update acquires a factor $(1 - \eta\lambda)$: every step shrinks the weights a little before the gradient moves them.

2. **Dropout.** Set a random fraction p of units to zero at each training step and rescale the survivors by $1/(1-p)$, so the expected activation is unchanged.

3. **Early stopping.** Halt at the validation minimum — the previous frames.

The econometric row

weight decay $\leftrightarrow$ **Ridge / LASSO**

dropout $\leftrightarrow$ **model averaging**

early stopping $\leftrightarrow$ **controlling specification search**

Two more, used without comment in practice: **batch normalisation**, which standardises activations within a batch, and **data augmentation**, which manufactures training examples by transformations that preserve the label.

Two Refinements Worth Knowing

Why AdamW, not Adam plus ℓ_2 . With plain SGD, adding $\frac{\lambda}{2} \|\theta\|^2$ to the loss and shrinking the weights each step are the *same* operation. **With Adam they are not:** the penalty's gradient is divided by the same running second moment as everything else, so parameters with large gradients are decayed *less*. AdamW decouples it.

Early stopping is Ridge. Stopping after T steps at rate η shrinks the solution about as much as a penalty $\lambda \approx 1/(\eta T)$ would.

Dropout, read as a Bayesian

Dropping units at random trains an ensemble of 2^m sub-networks with shared weights, and averages them at prediction time. That is **model averaging** — and under one reading, approximate variational Bayes. A bonus follows: leave dropout *on* at prediction time, sample repeatedly, and the spread is a usable **predictive-uncertainty band** — cheap, and worth having for policy evaluation. It is parameter uncertainty, not data noise; do not read it as a forecast interval.

Loshchilov & Hutter (2019); Srivastava et al. (2014); Gal & Ghahramani (2016); Goodfellow, Bengio & Courville (2016) §7.8. Feng & Miao, *AI for Economists*, ch. 9 §9.4.

Measured: What Regularisation Is Worth

Strategy	$n = 2,000$		$n = 50,000$	
	best ep.	best val.	best ep.	best val.
none	82	0.1535	39	0.0788
weight decay $1e-4$	82	0.1536	42	0.0804
dropout 0.1	145	0.1257	72	0.0780
dropout 0.2	122	0.1341	133	0.0792
wd $1e-4$ + dropout 0.1	145	0.1238	56	0.0780

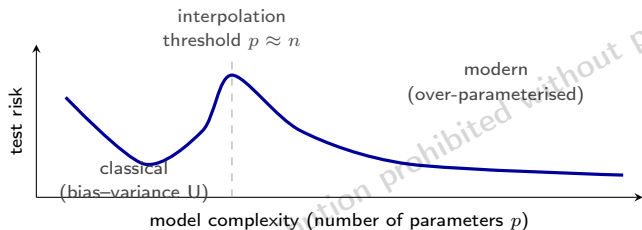
Regularisation pays where data is scarce and almost nowhere else. At $n = 2,000$ dropout buys 18% of validation loss; at $n = 50,000$ it buys 1%.

And two cautions this run supplies. Weight decay at 10^{-4} does essentially nothing here — and at $n = 50,000$ it is very slightly worse. More dropout is not better: 0.2 loses to 0.1 at both sample sizes.

Ledger regularization: same process, network and budget as above; median over three seeds. Dropout also pushes the best epoch from 82 to 145 — it does not prevent overfitting, it delays it. Code: [s07_generate.py](#), `run_regularization`.

The Modern Twist: Double Descent

The “validation turns up, therefore overfitting” story is only the **under-parameterised half** of the picture.



- **The interpolation threshold.** As p grows, test risk follows the classical U — until $p \approx n$, where the model fits the sample exactly and risk **peaks**.
- **The second descent.** Push past it, $p \gg n$: descent selects a **minimum-norm** interpolant, and test risk *falls again*.

More parameters than observations can help rather than hurt — which is not a licence to stop measuring, but is why the 1,377-parameter network of 7.B1 is not absurd on a problem with no data at all.

Where the “Implicit Regularisation” Comes From

Take the smallest case: fit **two** points with **three** parameters. Infinitely many parameter vectors pass through both, the loss is zero at every one, and it cannot choose.

So what does?

The optimizer does, and its preference is the ℓ_2 norm — of all exact fits, descent from zero returns the smallest-coefficient one: the **flattest** curve through the points.

- In least squares, descent from the origin converges *exactly* to the minimum- $\|w\|_2$ solution — the pseudo-inverse $w = \Phi^+ y$.
- In separable classification it converges in direction to the max-margin (again minimum-norm) predictor.

No penalty is ever written down — the dynamics of descent supply the bias.

The mechanism

From a near-zero start, descent only moves along directions the data excites. The “wiggly” null-space directions are never touched, so the solution stays small — and small norm means **smooth**, which is what powers the second descent. **The analogy:** Tikhonov regularisation with $\lambda \rightarrow 0^+$, selected by the algorithm rather than by you.

Why an Economist Should Care: the Blessings of Dimensionality

In forecasting. Liao, Ma, Neuhierl & Shi (2023) ask whether noise hurts economic forecasts. When the target is driven by *latent factors*, forecasting models are **not sparse**: adding many predictors — *even pure noise* — past the sample size *lowers* out-of-sample risk, beating PCA- and LASSO-style reduction on US inflation. Overfitting need not inflate out-of-sample variance.

In solving models. Ebrahimi Kahou & Fernández-Villaverde (2026) find the same double descent in networks that minimise **Euler and Bellman residuals**:

- **no overfitting** — as width grows, *off-grid* residuals keep falling toward the benchmark solution;
- **algorithmic stability** — wide networks concentrate across random seeds where narrow ones do not.

Measured on LQ, McCall and growth models.

Over-parameterisation is a feature. And note why the second result matters here: its “no overfitting” claim is about **off-grid Euler residuals** — exactly the validation instrument 7.B3 insists on.

Takeaways, and What Is Still Missing

- Backpropagation is the chain rule with the intermediate quantities named and reused — and those quantities are **shadow prices**.
- The optimizer is first-order because it must be: 10^{12} Hessian entries at $N = 10^6$. **Adam is SGD with per-parameter steps.**
- What is guaranteed is a *stationary point* under the Robbins–Monro conditions. 3.A2's contraction bound is gone.
- Measured here: a training loss **below the noise floor** is the overfitting diagnostic, and regularisation pays when data is scarce and almost nowhere else.

Bridge

You can now train a generic $\mathcal{N}(x; \theta)$ end to end. **Generic is not enough for economics.** 7.A4 asks which constraints on $\mathbf{W}$ a structured problem deserves; 7.B3 imposes the ones theory dictates — monotonicity, convexity — by construction rather than by hope.

And Block B asks the question this deck has carefully avoided: what do you train on when **there is no dataset?**